Escola Sesi/Senai de Referência

Pesquisa sobre APIs

Beatriz Fröhlich
Caio Henrique Leite
Carlos Eduardo Buttke
Lucas Onofre Senczuck
Nordeley Medeiros Corradini

Joinville
2026

SUMÁRIO

1. O que é uma API e qual é sua finalidade? 3
2. Como funciona a comunicação entre sistemas? 4
3. O que são APIs REST? 5
4. Métodos HTTP 6
    4.1. GET — Consultar dados 6
    4.2. POST — Cadastrar dados 6
    4.3. PUT — Substituir um recurso 7
    4.4. PATCH — Alterar parcialmente um recurso 7
    4.5. DELETE — Excluir dados 8
5. Requisições e respostas HTTP 9
6. O que são endpoints? 10
7. O formato JSON 11
8. Diferenças entre uma API e um sistema web tradicional 12
9. Como uma aplicação PHP pode consumir uma API? 13
10. Como uma API disponibiliza informações para outros sistemas? 15
11. Autenticação, autorização e segurança 16
12. Exemplos de APIs em sistemas empresariais 17
13. Conclusão 18
14. Fontes consultadas 19

1. O que é uma API e qual é sua finalidade?

API significa Application Programming Interface, ou Interface de Programação de Aplicações. Ela permite que diferentes sistemas e aplicações se comuniquem e compartilhem informações por meio de regras e métodos definidos.

Sua finalidade é facilitar a integração entre sistemas, permitindo que uma aplicação utilize funcionalidades ou dados disponibilizados por outra sem precisar conhecer todos os detalhes de seu funcionamento interno.

No Ferroramas, por exemplo, uma API poderia disponibilizar informações sobre trens, rotas, funcionários e sensores para outros sistemas de monitoramento ferroviário.

2. Como funciona a comunicação entre sistemas?

A comunicação acontece quando uma aplicação envia uma requisição para outra aplicação que disponibiliza uma API. O sistema que recebe a requisição processa a solicitação e retorna uma resposta.

Em uma API web, essa comunicação geralmente acontece por meio do protocolo HTTP ou HTTPS.

O processo pode ser dividido em três etapas:

1. O cliente envia uma requisição para um endpoint.
2. O servidor processa a solicitação e consulta ou modifica os dados, quando necessário.
3. O servidor retorna uma resposta com um código de status e, frequentemente, informações em JSON.

Por exemplo, um sistema externo poderia solicitar a lista de trens cadastrados no Ferroramas e receber os dados disponibilizados pelo servidor.

3. O que são APIs REST?

REST significa Representational State Transfer. É um estilo arquitetural utilizado na construção de serviços web que organizam os recursos por meio de identificadores, geralmente URLs, e utilizam métodos HTTP para realizar operações.

APIs REST normalmente trabalham com recursos como usuários, trens e rotas. Esses recursos podem ser consultados, cadastrados, alterados ou excluídos conforme as permissões disponíveis.

Um exemplo de organização seria:

- /api/usuarios: representa a coleção de usuários.
- /api/trens: representa a coleção de trens.
- /api/rotas: representa a coleção de rotas.
- /api/sensores: representa a coleção de sensores.

Uma API REST bem estruturada utiliza os métodos HTTP de maneira coerente e retorna códigos de status adequados.

4. Métodos HTTP

Os métodos HTTP indicam a operação que o cliente deseja realizar.

4.1. GET — Consultar dados

O método GET é utilizado para buscar informações, sem solicitar uma alteração no estado dos recursos.

Exemplo de requisição:

```
GET /api/trens HTTP/1.1
Host: exemplo.com
Accept: application/json
```

Exemplo de resposta:

```
HTTP/1.1 200 OK
Content-Type: application/json
[
  {
    "id": 1,
    "nome": "Trem 01",
    "status": "ativo"
  },
  {
    "id": 2,
    "nome": "Trem 02",
    "status": "manutencao"
  }
]
```

Nesse exemplo, o servidor retorna os trens cadastrados.

4.2. POST — Cadastrar dados

O método POST é utilizado para enviar informações ao servidor, geralmente para criar um novo recurso.

Exemplo de requisição:

```
POST /api/trens HTTP/1.1
Host: exemplo.com
Content-Type: application/json
{
  "nome": "Trem 03",
```

```
  "status": "ativo"
}
```

Exemplo de resposta:

```
HTTP/1.1 201 Created
Content-Type: application/json
{
  "id": 3,
  "nome": "Trem 03",
  "status": "ativo"
}
```

O código 201 indica que o recurso foi criado com sucesso.

4.3. PUT — Substituir um recurso

O método PUT é utilizado para substituir a representação de um recurso existente, conforme as regras da API.

Exemplo de requisição:

```
PUT /api/trens/3 HTTP/1.1
Host: exemplo.com
Content-Type: application/json
{
  "nome": "Trem 03",
  "status": "manutencao"
}
```

Nesse caso, os dados enviados representam a nova versão do recurso. Os campos necessários dependem do contrato definido pela API.

4.4. PATCH — Alterar parcialmente um recurso

O método PATCH é utilizado quando se deseja modificar apenas parte das informações de um recurso.

Exemplo:

```
PATCH /api/trens/3 HTTP/1.1
Host: exemplo.com
Content-Type: application/json
{
  "status": "ativo"
}
```

Nesse exemplo, somente o status do trem é alterado, sem a necessidade de reenviar os demais campos.

4.5. DELETE — Excluir dados

O método DELETE é utilizado para solicitar a exclusão de um recurso.

Exemplo:

```
DELETE /api/trens/3 HTTP/1.1
Host: exemplo.com
```

Uma resposta possível é:

```
HTTP/1.1 204 No Content
```

O código 204 indica que a operação foi concluída e que a resposta não possui conteúdo. A API também pode utilizar outro código adequado, conforme sua implementação.

5. Requisições e respostas HTTP

Uma requisição HTTP é uma mensagem enviada pelo cliente ao servidor. Ela pode conter o método, o caminho do recurso, cabeçalhos e, quando necessário, um corpo com dados.

A resposta HTTP é enviada pelo servidor e contém um código de status, cabeçalhos e, opcionalmente, um corpo com informações.

Alguns códigos de status comuns são:

- 200 OK: a solicitação foi processada com sucesso.
- 201 Created: um recurso foi criado.
- 204 No Content: a operação foi concluída sem conteúdo na resposta.
- 400 Bad Request: a requisição apresenta algum problema.
- 401 Unauthorized: faltam credenciais válidas ou a autenticação não foi aceita.
- 403 Forbidden: o acesso foi negado por falta de permissão.
- 404 Not Found: o recurso solicitado não foi encontrado.
- 500 Internal Server Error: ocorreu um erro interno no servidor.

Esses códigos ajudam o cliente a entender o resultado da operação e permitem que a aplicação trate erros de maneira adequada.

6. O que são endpoints?

Endpoints são os pontos de acesso disponibilizados por uma API. Normalmente, são identificados por uma URL ou por um caminho associado a um método HTTP.

Exemplos:

- GET /api/trens: consulta os trens.
- POST /api/trens: cadastra um trem.
- GET /api/trens/3: consulta o trem de ID 3.
- PATCH /api/trens/3: altera parte dos dados do trem de ID 3.
- DELETE /api/trens/3: solicita a exclusão do trem de ID 3.

O endpoint define onde a solicitação será direcionada. O método HTTP e os dados enviados ajudam a determinar qual operação será realizada.

7. O formato JSON

JSON significa JavaScript Object Notation. É um formato textual utilizado para representar e transmitir dados de maneira estruturada.

Ele permite organizar informações em objetos, listas, pares de chave e valor, números, textos, valores booleanos e valores nulos.

Exemplo:

```
{
  "id": 1,
  "nome": "Trem 01",
  "responsavel_id": 5,
  "ativo": true
}
```

Nesse exemplo, o trem possui um identificador, um nome, o ID do funcionário responsável e uma informação que indica se está ativo.

O JSON facilita a comunicação entre sistemas desenvolvidos com linguagens diferentes, pois possui uma estrutura simples e amplamente aceita.

8. Diferenças entre uma API e um sistema web tradicional

Um sistema web tradicional geralmente entrega páginas HTML para que o usuário navegue e interaja por meio do navegador. O servidor pode processar formulários, consultar o banco de dados e devolver páginas completas.

Uma API, por outro lado, normalmente disponibiliza dados ou funcionalidades para serem utilizados por outras aplicações. Em vez de retornar uma página HTML, ela pode retornar dados em JSON.

As duas abordagens não são excludentes. Uma aplicação PHP pode gerar as páginas de um sistema e também disponibilizar uma API para que aplicativos móveis ou outros sistemas consultem suas informações.

No Ferroramas, por exemplo, as telas administrativas podem ser páginas web tradicionais, enquanto uma API poderia disponibilizar informações sobre trens e sensores para um painel externo.

9. Como uma aplicação PHP pode consumir uma API?

Uma aplicação PHP pode consumir uma API enviando requisições HTTP. Uma das formas de fazer isso é utilizando a extensão cURL, que permite configurar a URL, o método HTTP, os cabeçalhos e os dados enviados.

Exemplo de consumo de uma API utilizando GET:

```
<?php

$url = 'https://api.exemplo.com/trens';

curl = curl_init(url);

curl_setopt_array($curl, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_TIMEOUT => 10,
    CURLOPT_HTTPHEADER => [
        'Accept: application/json'
    ]
]);

resposta = curl_exec(curl);
status = curl_getinfo(curl, CURLINFO_HTTP_CODE);

if ($resposta === false) {
    error_log(curl_error($curl));
    echo 'Não foi possível consultar a API.';
} elseif ($status >= 200 && $status < 300) {
    dados = json_decode(resposta, true);

    if (json_last_error() === JSON_ERROR_NONE) {
        echo '<pre>';
        print_r($dados);
        echo '</pre>';
    } else {
        echo 'A resposta da API não contém JSON válido.';
    }
} else {
    echo 'A API retornou um erro HTTP: ' . $status;
}

curl_close($curl);
```

O endereço utilizado é ilustrativo e precisa ser substituído pelo endereço real de uma API. O exemplo mostra como enviar uma requisição, verificar o resultado e converter o JSON em uma estrutura que pode ser utilizada pelo PHP.

10. Como uma API disponibiliza informações para outros sistemas?

Para disponibilizar informações, uma aplicação precisa implementar endpoints que recebam requisições, validem os dados, verifiquem as permissões e executem as operações necessárias.

Uma API desenvolvida em PHP pode consultar um banco de dados MySQL e retornar os resultados em JSON.

Por exemplo, um endpoint de consulta de trens poderia:

1. Receber uma requisição GET.
2. Verificar se o usuário possui autorização para consultar os dados.
3. Consultar os registros no banco de dados.
4. Converter os resultados para JSON.
5. Retornar a resposta com o código HTTP apropriado.

Dessa forma, outros sistemas poderiam utilizar as informações sem precisar acessar diretamente o banco de dados do Ferroramas.

11. Autenticação, autorização e segurança

A autenticação verifica a identidade de quem está acessando o sistema. A autorização verifica quais recursos e operações essa pessoa pode utilizar.

Em APIs, é importante implementar medidas de segurança como:

- Utilizar HTTPS para proteger os dados durante a transmissão.
- Verificar a identidade do usuário ou sistema cliente.
- Controlar as permissões de acesso a cada endpoint.
- Validar os dados recebidos antes de utilizá-los.
- Utilizar consultas preparadas ao acessar o banco de dados.
- Não expor senhas, tokens ou informações confidenciais nas respostas.
- Armazenar senhas utilizando funções apropriadas para hash de senhas.
- Limitar tentativas e registrar eventos relevantes de segurança.
- Tratar erros sem revelar detalhes internos do servidor.

Uma API não deve considerar que um usuário está autorizado apenas porque ele conhece o endereço de um endpoint. As permissões precisam ser verificadas no servidor em todas as operações protegidas.

No Ferroramas, um usuário comum poderia ter permissão para consultar determinadas informações, enquanto o administrador teria acesso às operações administrativas. Essa regra precisaria ser aplicada no backend, e não somente na interface.

12. Exemplos de APIs em sistemas empresariais

As APIs são utilizadas em diferentes áreas de uma empresa.

Sistemas financeiros: permitem integrar plataformas de gestão com serviços de pagamento e consulta de transações.

Comércio eletrônico: possibilitam consultar produtos, calcular fretes, acompanhar pedidos e integrar meios de pagamento.

Logística e transporte: permitem compartilhar informações sobre veículos, trajetos, entregas e localização.

Recursos humanos: facilitam a integração entre sistemas de funcionários, folha de pagamento e controle de acesso.

Monitoramento industrial: permitem disponibilizar leituras de sensores, estados de equipamentos e alertas para painéis de monitoramento.

No Ferroramas, uma API poderia permitir que um painel externo consultasse o estado dos trens, as rotas cadastradas e as informações dos sensores, respeitando as permissões e as medidas de segurança estabelecidas.

13. Conclusão

As APIs são importantes para a integração entre sistemas porque permitem compartilhar dados e funcionalidades de maneira padronizada. Com os métodos HTTP, os endpoints e o formato JSON, diferentes aplicações podem trocar informações sem depender da mesma linguagem de programação ou da mesma interface.

No Ferroramas, o conhecimento sobre APIs pode contribuir para futuras melhorias, como a integração com painéis de monitoramento, aplicativos externos e sistemas de acompanhamento ferroviário. Para que essa integração seja confiável, é necessário implementar validações, autenticação, autorização e outras medidas de segurança.

14. Referencias

FIELDING, Roy Thomas. Architectural styles and the design of network-based software architectures. Irvine: University of California, 2000. Disponível em: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm. Acesso em: 20 out. 2026.

MDN WEB DOCS. Códigos de status de resposta HTTP. [S. l.], [s. d.]. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Status. Acesso em: 20 out. 2026.

MDN WEB DOCS. Métodos de requisição HTTP. [S. l.], [s. d.]. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Methods. Acesso em: 20 out. 2026.

MDN WEB DOCS. Uma introdução ao JSON. [S. l.], [s. d.]. Disponível em: https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Core/Scripting/JSON. Acesso em: 20 out. 2026.

PHP. cURL. [S. l.], [s. d.]. Disponível em: https://www.php.net/manual/pt_BR/book.curl.php. Acesso em: 20 out. 2026.

PHP. json_decode. [S. l.], [s. d.]. Disponível em: https://www.php.net/manual/pt_BR/function.json-decode.php. Acesso em: 20 out. 2026.